5×5 围棋 AI 搜索算法实验报告

目录

1 项目概览 **2**

1.1 项目结构 2

1.2 GUI设计 2

2 围棋Agent **3**

2.1 random Agent 3

2.2 MCTS Agent 4

2.3 Minimax Agent 5

2.4 策略库 5

3 实验内容 **5**

3.1 实验设计 5

3.2 实验规则说明 6

3.3 MCTS 优化实验 6

3.4 MCTS vs Minimax 实验 7

4 实验现象分析与可能的解释 **9**

5 总结 **9**

6 AI辅助声明 **10**

1 项目概览

本项目是作业一的围棋 AI 任务，仓库链接：https://github.com/c1dn/ai-course-hw1。实验主要在 5×5 棋盘上完成；Agent 的开局策略、实验参数和启发式调参也都以 5×5 为目标。根据 5×5 围棋的已知结论，黑棋首手下天元 (3,3) 是最优开局，因此项目在公共开局策略中固定了这一手，以减少开局随机性对实验结果的影响。

1.1 项目结构

仓库核心结构如下：

- `dlgo/`：围棋规则框架，包含棋盘、棋串、合法性检查、劫判定、终局判断、地盘评估和胜负计算。
- `agents/random_agent.py`：随机智能体，从合法动作中均匀采样，用作弱基线和流程验证。
- `agents/mcts_agent.py`：MCTS 智能体，实现选择、扩展、模拟、反向传播四步，并支持多种优化开关。
- `agents/minimax_agent.py`：Minimax + Alpha-Beta 智能体，带走子排序、静态评估和轻量置换表。
- `agents/policy/`：策略库，包含 5×5 天元开局、MCTS 采样/扩展策略、Minimax 估值/排序策略。
- `gui/` 与 `play_gui.py`：基于 PySide6 的交互式对弈界面。
- `experiments/`：批量实验入口、实验计划和结果汇总。
- `report/`：本报告、图表生成脚本和报告插图。

1.2 GUI设计

GUI 入口为：`python play_gui.py`

界面采用左右分栏：左侧是棋盘绘制区域，右侧是对局配置、操作按钮、状态面板和走子日志。主要功能包括：

- 模式选择：Human vs AI、AI vs AI。
- Agent 选择：Random、MCTS、Minimax。
- 参数设置：MCTS rounds、Minimax depth。

- 对局操作：New Game、Pass、Undo、Resign。
- 状态显示：当前回合、对局结果、手数、局面分数估计、走子日志。
- 实验性裁定选项：启用后到达 `board_size * board_size * 2` 手时按当前分数裁定。

GUI 中的 MCTS Rounds：当 Rounds 设为 `-1` 时，并不是无限搜索，而是进入 auto rounds 模式。对于 5×5 棋盘，将设定为 600 rounds；同时模拟会走到终局。Rounds 为正数时，固定使用 `max_rollout_depth=28`。

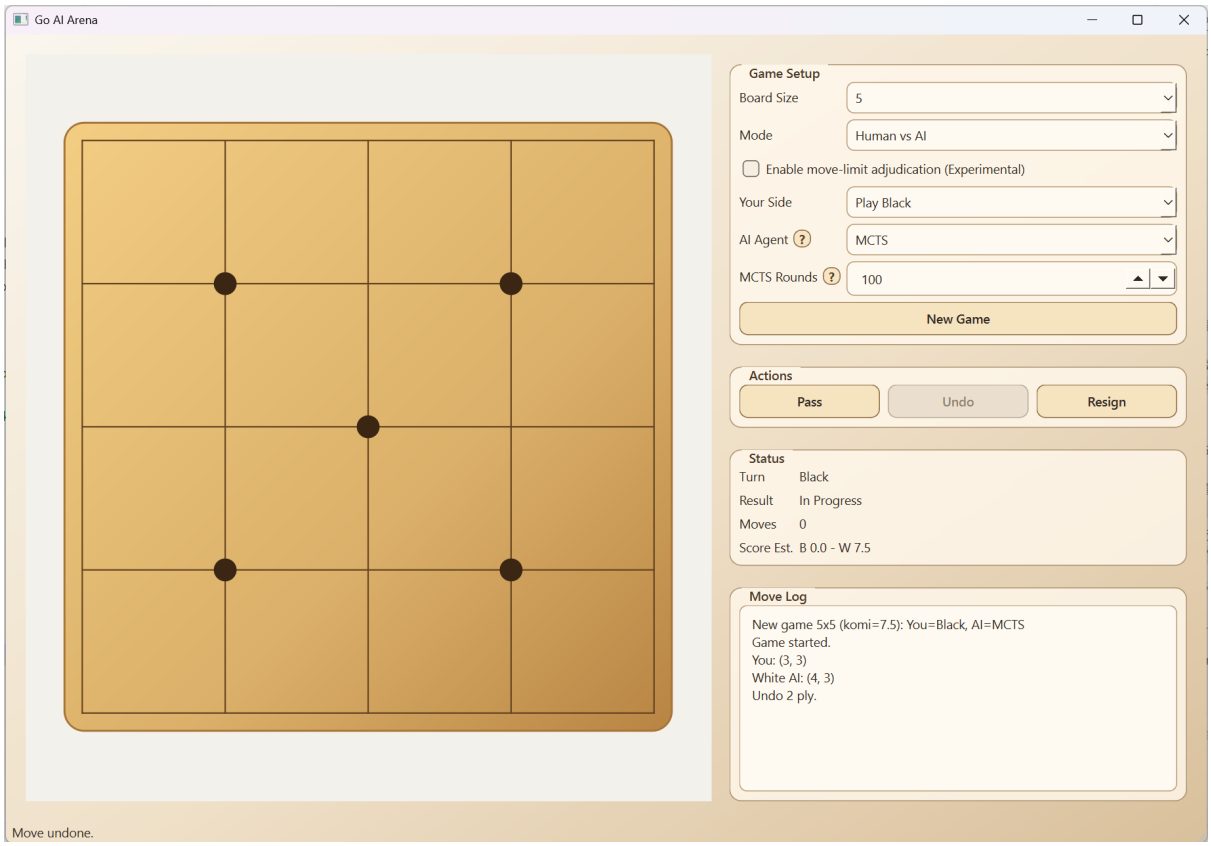

图 1: 围棋 AI 对弈 GUI 主界面截图

2 围棋Agent

2.1 random Agent

`agents/random_agent.py` 中的 Random Agent 从所有合法动作中随机选择一手。默认构造参数 `allow_resign=False`，此时不会主动认输；如果没有合法落子，则返回 pass。

和其他 Agent 一样，Random Agent 会先调用公共开局策略。如果当前局面是 5×5 空棋盘且轮到黑棋，第一手固定返回天元 `(3,3)`。

2.2 MCTS Agent

`agents/mcts_agent.py` 实现了蒙特卡洛树搜索。标准流程为：

1. **选择**：从根节点出发，沿搜索树选择得分最高的子节点。
2. **扩展**：在尚未完全展开的节点上扩展一个新动作。
3. **模拟**：从扩展后的局面继续进行模拟对局，直到终局或达到模拟深度上限。
4. **反向传播**：把模拟结果沿路径反向传播，更新访问次数和累计价值。

代码中标准 MCTS 构造函数：`MCTSAgent.standard_baseline()`，默认在模拟阶段随机选择合法落子；不加入启发式扩展和先验奖励；不提前截断，一直走到自然终局。

实验中的增强版 MCTS 默认使用如下配置：

- `exploration_weight=0.6`
- `max_rollout_depth=10`
- `rollout_policy=heuristic`
- `expansion_policy=heuristic`
- `use_prior_bonus=True`

`exploration_weight=0.6` 表示降低探索强度，让搜索更集中在当前估值较高的分支上； `max_rollout_depth=10` 表示模拟最多向后走 10 手，超过后用静态评估函数估计局面； `rollout_policy=heuristic` 表示模拟阶段不完全随机，而是优先采样提子、救棋、连接和中心等较合理的走法； `expansion_policy=heuristic` 表示扩展新节点时优先选择启发式评分较高的候选动作； `use_prior_bonus=True` 表示在选择阶段加入先验奖励，使搜索初期更快关注静态启发式认为重要的分支。与标准 MCTS 比较，增强版 MCTS 主要通过启发式引导和提前截断来提高搜索速度并降低随机模拟的噪声。

具体到选择阶段，子节点得分在 UCT 的基础上额外加入先验奖励：

$$\text{score} = Q + c\sqrt{\frac{\ln N}{n}} + 0.15 \cdot p \cdot \frac{\sqrt{N}}{n+1},$$

其中 Q 是当前子节点平均价值，N 是父节点访问次数，n 是子节点访问次数，p 是由静态启发式给出的先验。先验奖励的作用是让搜索初期更快关注提子、救棋、连接、中心和局部应手等高优先级动作。

模拟阶段则负责把叶子节点继续向后推进。启发式策略会先对落子打分，再在前 16 个候选中按权重采样。若达到最大模拟深度但棋局未结束，则调用静态评估函数估计近似胜率，避免长随机模拟耗时过长。

2.3 Minimax Agent

`agents/minimax_agent.py` 实现了固定深度 Minimax 搜索，并加入 Alpha-Beta 剪枝。其搜索过程以当前执棋方为根玩家，终局胜负返回 ± 10000，保证明确胜负优先于普通静态估值。

静态评估函数位于 `agents/policy/minimax_policy.py`，形式为：

$$E(s) = 0.7 \cdot \big(B - (W + \text{komi})\big) + \text{string_feature_margin}(s),$$

其中 B 表示黑方子数加黑地，W 表示白方子数加白地，`komi` 为贴目。棋串特征项主要考虑：

- 棋串大小和气数。
- 1 气、2 气弱棋惩罚。
- 气数充足棋串的稳定性奖励。
- 邻接同色棋串形成连接形状时的奖励。

为了控制 Minimax 的分支数量，搜索前会先对候选走法做启发式排序，优先考虑提子、救吃、连接、压迫和中心偏好，并惩罚明显的自填气走法。每个局面只保留评分最高的前 12 个动作继续递归搜索，从而在 5×5 小棋盘上提高 Alpha-Beta 剪枝效率。

2.4 策略库

策略库位于 `agents/policy/`，将规则和启发式从具体搜索算法中拆出。

- `opening_policy.py`：实现 5×5 黑棋首手天元 `(3,3)`。这是项目中所有 Agent 共用的开局规则。
- `minimax_policy.py`：提供 Minimax 静态评估、棋串特征、走子排序和局部战术打分。
- `mcts_policy.py`：提供 MCTS 的候选动作集合、启发式扩展、模拟采样、先验计算和模拟截断评估。

3 实验内容

3.1 实验设计

- **MCTS 优化实验**：固定 1000 rounds，比较不同 MCTS 优化项相对标准 MCTS 的收益。结果目录为 `experiments/results/mcts_ablation_plan/`。

- **MCTS vs Minimax 实验**：比较增强版 MCTS 与 Minimax 在不同 rounds 和 depth 下的表现。结果目录为 `experiments/results/experiment_plan/`。

每个配置运行 10 局：双方各执黑 5 局、执白 5 局。汇总胜率、平均总时长、平均手数、每步平均耗时、最大单步耗时，以及终局类型。

3.2 实验规则说明

所有实验均采用以下规则。

棋盘与开局 棋盘固定为 5×5。所有 Agent 在空棋盘黑棋先手时都会先走天元 (3,3)。这一设置基于 5×5 围棋中黑棋天元开局已被证明最优的结论，用于减少开局波动，使实验更集中地比较中后盘搜索能力。

贴目设置 默认贴目为 7.5。由于贴目为半目，实验中不会出现按分数完全平局的情况。

50 手裁定 实验脚本在 5×5 棋盘上达到 50 手而尚未自然终局时，不会继续无限运行，而是按当前棋盘调用同一套计分函数直接裁定，终局类型记为 50 手裁定终局。

原因是 MCTS 对 MCTS 或高模拟轮数 MCTS 对 Minimax 时，双方可能在小棋盘后期反复填子或进入其它耗时较长的收束阶段。 50 手相当于棋盘点数的两倍，已经足够覆盖大量落子、提子和收官行为；继续无上限运行对比较搜索算法的主要结论帮助有限，但会显著增加实验耗时。

3.3 MCTS 优化实验

MCTS 消融实验的对照组是参考 MCTS：

- 进行 1000 轮模拟。
- 模拟阶段随机选择合法落子，截断 50 手。
- 扩展阶段在候选动作中均匀选择，不使用启发式排序。
- 选择阶段不加入先验奖励。

各变体每组 10 局，对手均为参考 MCTS。结果如下：

表 1: 1000 rounds MCTS 消融实验

变体	变体胜率(%)	平均总时长(s)	变体每步(s)	参考每步(s)	平均胜负差	终局概况
参考 MCTS	50	324.51	10.44	10.23	0.8	10 次两连 pass
低探索系数	70	330.11	9.73	10.21	4.3	10 次两连 pass
截断 10 手 + 静态评估	100	223.90	3.21	12.71	6.3	10 次两连 pass
启发式模拟	80	411.69	16.95	9.46	4.4	9 次两连 pass, 1 次裁定
启发式扩展	70	314.76	11.55	10.61	2.8	10 次两连 pass
先验奖励	70	416.45	9.76	9.16	15.2	10 次两连 pass
完整增强版	90	340.92	5.07	10.88	21.9	10 次两连 pass

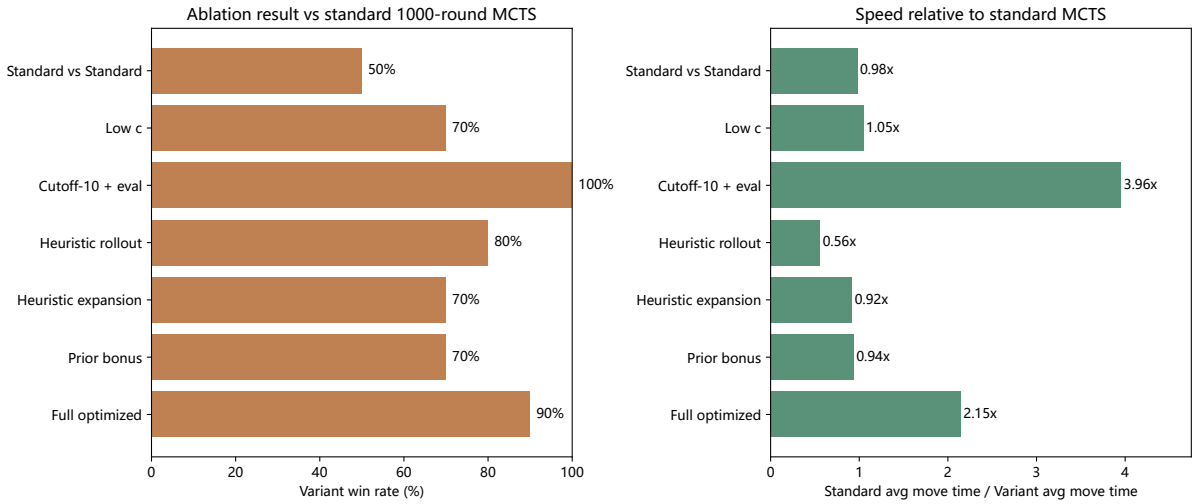

图 2: 1000 rounds 消融实验中各优化项相对参考 MCTS 的胜率与速度收益

结果显示，单项收益最明显的是“截断 10 手 + 静态评估”：它 10 局全胜，而且平均每步耗时显著低于参考 MCTS。说明在 5×5 小棋盘上，随机模拟走得太深不一定更好；更早截断模拟并调用静态评估，既减少噪声，也节省时间。

完整增强版的胜率为 90%，平均胜负差最大，并且每步耗时约为参考版的一半。它不是单项胜率最高，但棋力、速度和胜负差的综合表现最好。

3.4 MCTS vs Minimax 实验

MCTS vs Minimax 实验使用增强版 MCTS，与不同深度的 Minimax 对弈。

表 2: 组 1：MCTS 与 Minimax 同步提高预算

配置	MCTS 胜率(%)	平均总时长(s)	平均步数	MCTS 每步(s)	Minimax 每步(s)	平均 MCTS 胜负差
MCTS(300) vs Minimax(4)	0	51.08	34.5	2.09	0.91	-10.3
MCTS(600) vs Minimax(5)	0	123.86	33.9	4.29	3.21	-9.3
MCTS(2200) vs Minimax(6)	0	309.09	30.3	11.68	8.85	-6.9

同步提高双方预算后，MCTS 的平均胜负差有所缩小，但 30 局中仍然没有获胜。这说明在当前实现和 5×5 设定下，Minimax 的确定性搜索仍然更有效。

表 3: 组 2：固定 MCTS(2200)，扫描 Minimax 深度

配置	MCTS 胜率(%)	平均总时长(s)	平均步数	MCTS 每步(s)	Minimax 每步(s)	平均 MCTS 胜负差	Minimax 超 20s 局数
MCTS(2200) vs Minimax(4)	0	237.53	33.0	13.72	0.85	-8.2	0
MCTS(2200) vs Minimax(5)	0	297.90	43.6	11.47	2.50	-19.5	0
MCTS(2200) vs Minimax(6)	0	309.09	30.3	11.68	8.85	-6.9	10

固定 MCTS(2200) 后，即使 Minimax 只搜到 depth=4，也能稳定压制 MCTS。depth=6 的 Minimax 明显更慢，10 局都出现过单步超过 20 秒的情况，但仍保持全胜。

表 4: 组 3：固定 Minimax(5)，扫描 MCTS rounds

配置	MCTS 胜率(%)	平均总时长(s)	MCTS 每步(s)	Minimax 每步(s)	MCTS 最大单步(s)	平均 MCTS 胜负差	50 手裁定局数	MCTS 超 20s 局数
MCTS(2200) vs Minimax(5)	0	297.90	11.47	2.50	40.40	-19.5	1	10
MCTS(3000) vs Minimax(5)	10	405.81	14.62	2.33	54.94	-20.0	2	10
MCTS(4000) vs Minimax(5)	50	484.95	19.92	2.15	72.90	5.5	5	10
MCTS(5000) vs Minimax(5)	40	551.26	24.11	2.05	93.33	0.5	3	10

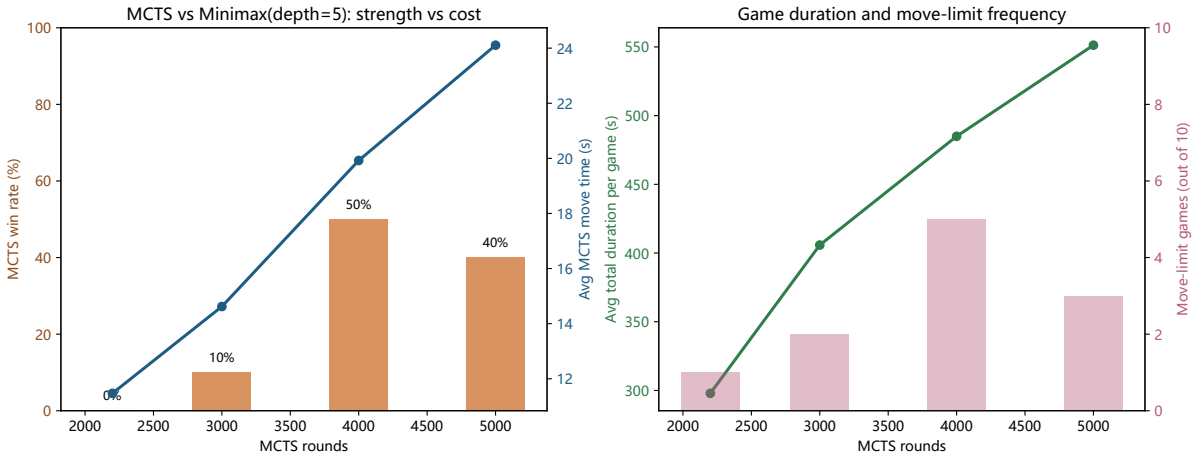

图 3: 固定 `Minimax(depth=5)` 时，MCTS rounds 对胜率、耗时和 50 手裁定的影响

当 Minimax 固定为 depth=5 时，MCTS rounds 提升到 4000 后首次达到 50% 胜率，但代价很高：平均每步接近 20 秒，最大单步达到 72.90 秒。继续增加到 5000 rounds 后胜率反而回落到 40%，说明单纯增加 rounds 的收益并不稳定。

表 5: 高 rounds 配置下的颜色拆分结果

配置	MCTS 执黑胜率	MCTS 执白胜率	备注
MCTS(2200) vs Minimax(5)	0/5	0/5	仅白棋子实验出现 1 局 50 手裁定
MCTS(3000) vs Minimax(5)	0/5	1/5	唯一胜局来自白棋
MCTS(4000) vs Minimax(5)	0/5	5/5	全部胜局都来自白棋，且全为 50 手裁定
MCTS(5000) vs Minimax(5)	0/5	4/5	白棋胜局中 3 局为 50 手裁定

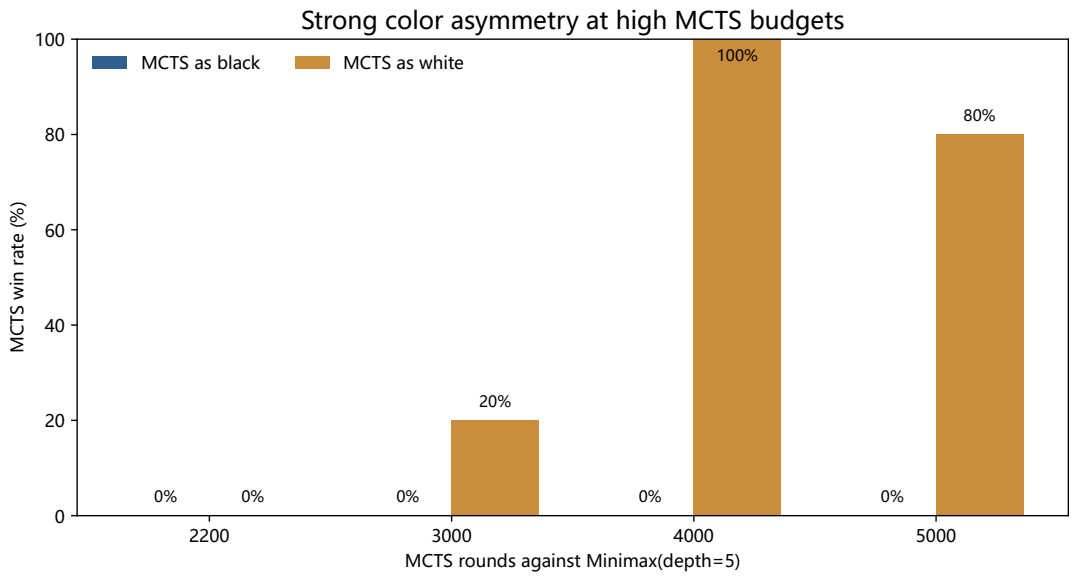

图 4: 高模拟轮数 MCTS 对战 `Minimax(depth=5)` 时的执黑/执白胜率差异

4 实验现象分析与可能的解释

(1) **模拟截断与启发式优化显著提高 MCTS 效率。** MCTS 对照组使用较长的随机模拟，既耗时，又会引入较大噪声。而在小棋盘上，较浅模拟后启发式评估比继续淡出随机模拟更可靠。

(2) **Minimax 在 5×5 上更占优势。** 5×5 棋盘小，局部吃子、救吃和连接变化较集中。 Minimax 通过走子排序和 Alpha-Beta 剪枝，可以把有限预算集中到关键确定性变化上； MCTS 即使加入启发式，仍需要用大量模拟估计动作价值，单位时间效率较低。而在大棋盘上，搜索树规模会快速增长，Minimax 的固定深度搜索容易出现组合爆炸， MCTS 这种采样式搜索反而能更好合理分配资源。

5 总结

本项目完成了 5×5 围棋中的 Random、MCTS、Minimax、公共策略库、GUI 和批量实验脚本。结合实验结果，可以得到两点结论：

- 在 5×5、7.5 贴目、黑棋首手天元的设定下，Minimax 是更稳定、单位时间效率更高的强基线。
- MCTS 优化中最关键的是模拟截断加静态评估，能够同时提升胜率和速度。

6 AI辅助声明

本项目在代码整理和实验过程中使用了 AI 辅助工具 Codex。

主要使用内容

- 编写 GUI。
- 将 Agent 中的启发式策略整理为独立策略库。
- 编写和统一代码注释。
- 生成批量实验脚本。
- 整理实验结果，生成实验结果图表脚本。

使用感受 它非常适合做工程整理、脚本生成和重复性检查，能明显减少手工搬运数据与查找不一致的时间；但在算法抉择、实验设计、解释现象仍然需要人工判断。